

Plano de Desenvolvimento — Trabalho Prático de Engenharia de Dados usando Java

1. Visão Geral do Projeto

Este projeto será desenvolvido em **Java**, utilizando **Spring Boot** como framework principal para a construção da aplicação back-end. A aplicação terá como objetivo implementar operações de CRUD em bancos de dados relacionais e não relacionais, além de preparar a integração de dados em um modelo analítico baseado em esquema estrela.

O trabalho será dividido em três partes:

1. CRUD Relacional com PostgreSQL.
2. CRUD NoSQL com MongoDB.
3. Integração de Dados com esquema estrela e Apache Hop.

A escolha do Java com Spring Boot permite organizar o projeto em camadas, facilitar a conexão com PostgreSQL e MongoDB, melhorar a manutenção do código e tornar a apresentação mais clara para avaliação.

2. Stack Tecnológica

2.1 Linguagem e Framework

- Java 21 ou superior
- Spring Boot
- Maven

2.2 Bibliotecas principais

- Spring Web
- Spring Data JPA
- Spring Data MongoDB
- PostgreSQL Driver
- Bean Validation
- Lombok, opcional
- Spring Boot DevTools, opcional
- Flyway, opcional, para versionamento do banco relacional

2.3 Bancos de Dados

- PostgreSQL hospedado na AWS RDS
- MongoDB hospedado na AWS

2.4 ETL

- Apache Hop

2.5 Ferramentas de apoio

- Git
- GitHub
- Postman ou Insomnia
- DBeaver ou pgAdmin
- MongoDB Compass
- VS Code, IntelliJ IDEA ou Eclipse

3. Arquitetura Geral do Sistema

A aplicação Java será organizada em camadas:

```
Controller → Service → Repository → Database
```

3.1 Controller

Responsável por receber as requisições HTTP e retornar respostas para o cliente.

Exemplo:

```
UsuarioController  
EstudanteController  
CursoController  
VinculoController
```

3.2 Service

Responsável pelas regras de negócio, validações e chamadas aos repositórios.

Exemplo:

```
UsuarioService  
EstudanteService  
CursoService  
VinculoService
```

3.3 Repository

Responsável pela comunicação com o banco de dados.

Para PostgreSQL:

```
JpaRepository
```

Para MongoDB:

```
MongoRepository
```

3.4 Model / Entity / Document

Responsável por representar as entidades do sistema.

No PostgreSQL, as classes serão mapeadas com JPA.

No MongoDB, as classes serão mapeadas como documentos.

3.5 DTOs

Responsáveis por transportar dados entre cliente e aplicação.

Exemplo:

```
UsuarioRequestDTO  
UsuarioResponseDTO  
CursoRequestDTO  
CursoResponseDTO
```

4. Estrutura Sugerida do Projeto

```
engenharia-dados-java/  
├── backend/  
│   ├── pom.xml  
│   └── src/  
│       ├── main/  
│       │   ├── java/  
│       │   │   ├── br/  
│       │   │   │   ├── ufs/  
│       │   │   │   └── engenhariadados/  
│       │   └── resources/  
│       └── test/  
└──
```

```

├── EngenhariaDadosApplication.java
├── config/
├── controller/
│   ├── UsuarioController.java
│   ├── EstudanteController.java
│   ├── CursoController.java
│   └── VinculoController.java
├── dto/
├── exception/
├── model/
│   ├── relacional/
│   └── nosql/
├── repository/
│   ├── relacional/
│   └── nosql/
├── service/
├── resources/
│   ├── application.properties
│   └── db/
│       └── migration/
├── test/
├── database/
│   ├── postgresql/
│   │   ├── schema.sql
│   │   ├── seed.sql
│   │   └── star_schema.sql
│   └── mongodb/
│       ├── collections.js
│       └── validators.js
├── etl/
│   ├── pipelines/
│   └── workflows/
├── data/
│   ├── unidades_academicas/
│   ├── componentes_curriculares/
│   ├── docentes/
│   └── turmas/
├── docs/
│   ├── plano-desenvolvimento.md
│   ├── mapeamento-nosql.md
│   ├── esquema-estrela.md
│   ├── apache-hop.md
│   ├── evidencias.md
│   └── relatorio.md
├── README.md
└── .gitignore

```

5. Parte 1 — CRUD Relacional com Java, Spring Boot e PostgreSQL

5.1 Objetivo

Desenvolver uma aplicação Java que realize operações de CRUD em um banco de dados PostgreSQL hospedado na AWS.

As tabelas trabalhadas serão:

- usuário;
- estudante;
- curso;
- vínculo.

5.2 Banco de Dados PostgreSQL

O banco PostgreSQL deverá conter todas as tabelas do esquema relacional trabalhado em aula.

Exemplo de tabelas:

```
CREATE TABLE usuario (  
  id_usuario SERIAL PRIMARY KEY,  
  nome VARCHAR(120) NOT NULL,  
  email VARCHAR(120) NOT NULL UNIQUE,  
  senha VARCHAR(255) NOT NULL,  
  tipo_usuario VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE estudante (  
  id_estudante SERIAL PRIMARY KEY,  
  matricula VARCHAR(30) NOT NULL UNIQUE,  
  id_usuario INTEGER NOT NULL,  
  data_nascimento DATE,  
  CONSTRAINT fk_estudante_usuario  
    FOREIGN KEY (id_usuario)  
    REFERENCES usuario(id_usuario)  
);
```

```
CREATE TABLE curso (  
  id_curso SERIAL PRIMARY KEY,  
  nome VARCHAR(150) NOT NULL,  
  codigo VARCHAR(30) NOT NULL UNIQUE,  
  carga_horaria INTEGER  
);
```

```
CREATE TABLE vinculo (
    id_vinculo SERIAL PRIMARY KEY,
    id_estudante INTEGER NOT NULL,
    id_curso INTEGER NOT NULL,
    data_inicio DATE NOT NULL,
    status VARCHAR(30) NOT NULL,
    CONSTRAINT fk_vinculo_estudante
        FOREIGN KEY (id_estudante)
        REFERENCES estudante(id_estudante),
    CONSTRAINT fk_vinculo_curso
        FOREIGN KEY (id_curso)
        REFERENCES curso(id_curso)
);
```

5.3 Configuração do Spring Boot com PostgreSQL

No arquivo `application.properties`:

```
spring.datasource.url=jdbc:postgresql://HOST_AWS:5432/NOME_BANCO
spring.datasource.username=USUARIO
spring.datasource.password=SENHA

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.flyway.enabled=true
```

5.4 Entidades JPA

Exemplo de entidade `Usuario`:

```
@Entity
@Table(name = "usuario")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_usuario")
    private Long id;

    @Column(nullable = false, length = 120)
    private String nome;

    @Column(nullable = false, unique = true, length = 120)
    private String email;
```

```

@Column(nullable = false, length = 255)
private String senha;

@Column(name = "tipo_usuario", nullable = false, length = 50)
private String tipoUsuario;
}

```

5.5 Repositórios JPA

```

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    boolean existsByEmail(String email);
}

```

5.6 Serviços

A camada de serviço deverá conter regras como:

- verificar duplicidade de e-mail;
- verificar se estudante existe antes de criar vínculo;
- verificar se curso existe antes de criar vínculo;
- validar campos obrigatórios;
- lançar exceções quando registros não forem encontrados.

Exemplo:

```

@Service
public class UsuarioService {

    private final UsuarioRepository usuarioRepository;

    public UsuarioService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    public Usuario criar(Usuario usuario) {
        if (usuarioRepository.existsByEmail(usuario.getEmail())) {
            throw new RuntimeException("E-mail já cadastrado.");
        }

        return usuarioRepository.save(usuario);
    }

    public List<Usuario> listar() {
        return usuarioRepository.findAll();
    }

    public Usuario buscarPorId(Long id) {
        return usuarioRepository.findById(id)
    }
}

```

```

        .orElseThrow(() -> new RuntimeException("Usuário não
encontrado."));
    }

    public Usuario atualizar(Long id, Usuario dados) {
        Usuario usuario = buscarPorId(id);

        usuario.setNome(dados.getNome());
        usuario.setEmail(dados.getEmail());
        usuario.setSenha(dados.getSenha());
        usuario.setTipoUsuario(dados.getTipoUsuario());

        return usuarioRepository.save(usuario);
    }

    public void remover(Long id) {
        Usuario usuario = buscarPorId(id);
        usuarioRepository.delete(usuario);
    }
}

```

5.7 Controllers REST

Exemplo:

```

@RestController
@RequestMapping("/api/usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @PostMapping
    public ResponseEntity<Usuario> criar(@RequestBody Usuario usuario) {
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(usuarioService.criar(usuario));
    }

    @GetMapping
    public ResponseEntity<List<Usuario>> listar() {
        return ResponseEntity.ok(usuarioService.listar());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Usuario> buscarPorId(@PathVariable Long id) {
        return ResponseEntity.ok(usuarioService.buscarPorId(id));
    }
}

```

```

    }

    @PutMapping("/{id}")
    public ResponseEntity<Usuario> atualizar(
        @PathVariable Long id,
        @RequestBody Usuario usuario
    ) {
        return ResponseEntity.ok(usuarioService.atualizar(id, usuario));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> remover(@PathVariable Long id) {
        usuarioService.remover(id);
        return ResponseEntity.noContent().build();
    }
}

```

5.8 Endpoints da Parte 1

Usuários

```

POST /api/usuarios
GET /api/usuarios
GET /api/usuarios/{id}
PUT /api/usuarios/{id}
DELETE /api/usuarios/{id}

```

Estudantes

```

POST /api/estudantes
GET /api/estudantes
GET /api/estudantes/{id}
PUT /api/estudantes/{id}
DELETE /api/estudantes/{id}

```

Cursos

```

POST /api/cursos
GET /api/cursos
GET /api/cursos/{id}
PUT /api/cursos/{id}
DELETE /api/cursos/{id}

```

Vínculos

```
POST /api/vinculos
GET /api/vinculos
GET /api/vinculos/{id}
PUT /api/vinculos/{id}
DELETE /api/vinculos/{id}
```

6. Parte 2 — CRUD NoSQL com Java, Spring Boot e MongoDB

6.1 Objetivo

Mapear as tabelas relacionais para coleções MongoDB e implementar operações de CRUD usando Java com Spring Data MongoDB.

As estruturas mapeadas serão:

- usuários;
- estudantes;
- cursos;
- vínculos.

6.2 Configuração do MongoDB

No arquivo `application.properties`:

```
spring.data.mongodb.uri=mongodb://USUARIO:SENHA@HOST_AWS:27017/NOME_BANCO
```

6.3 Documentos MongoDB

UsuarioDocument

```
@Document(collection = "usuarios")
public class UsuarioDocument {

    @Id
    private String id;

    private String nome;
    private String email;
    private String senha;
```

```
        private String tipoUsuario;  
    }  
}
```

EstudanteDocument

```
@Document(collection = "estudantes")  
public class EstudanteDocument {  
  
    @Id  
    private String id;  
  
    private String matricula;  
    private String usuarioId;  
    private LocalDate dataNascimento;  
  
}
```

CursoDocument

```
@Document(collection = "cursos")  
public class CursoDocument {  
  
    @Id  
    private String id;  
  
    private String nome;  
    private String codigo;  
    private Integer cargaHoraria;  
  
}
```

VinculoDocument

```
@Document(collection = "vinculos")  
public class VinculoDocument {  
  
    @Id  
    private String id;  
  
    private String estudanteId;  
    private String cursoId;  
    private LocalDate dataInicio;  
    private String status;  
  
}
```

6.4 Repositórios MongoDB

```
public interface UsuarioMongoRepository extends
MongoRepository<UsuarioDocument, String> {
    boolean existsByEmail(String email);
}
```

```
public interface EstudanteMongoRepository extends
MongoRepository<EstudanteDocument, String> {
    boolean existsByMatricula(String matricula);
}
```

```
public interface CursoMongoRepository extends MongoRepository<CursoDocument,
String> {
    boolean existsByCodigo(String codigo);
}
```

```
public interface VinculoMongoRepository extends
MongoRepository<VinculoDocument, String> {
}
```

6.5 Validação das Restrições no MongoDB

Como o MongoDB não trabalha com chaves estrangeiras da mesma forma que o PostgreSQL, algumas restrições devem ser garantidas pela aplicação.

6.5.1 Restrição de chave

- Usar `_id` como identificador único.
- Criar índices únicos para campos como `email`, `matricula` e `codigo`.

6.5.2 Integridade referencial

Antes de cadastrar um estudante:

- verificar se o usuário existe.

Antes de cadastrar um vínculo:

- verificar se o estudante existe;
- verificar se o curso existe.

6.5.3 Domínio

Validar campos como:

- tipo de usuário;
- status do vínculo;
- carga horária;
- e-mail;
- matrícula.

6.5.4 NOT NULL

Usar validações no Java:

```
@NotBlank
@NotNull
>Email
```

Exemplo:

```
public class UsuarioRequestDTO {

    @NotBlank
    private String nome;

    @NotBlank
    @Email
    private String email;

    @NotBlank
    private String senha;

    @NotBlank
    private String tipoUsuario;
}
```

6.6 Endpoints da Parte 2

Para evitar conflito com o CRUD relacional, os endpoints NoSQL podem receber o prefixo `/mongo`.

Usuários MongoDB

```
POST /api/mongo/usuarios
GET /api/mongo/usuarios
GET /api/mongo/usuarios/{id}
```

```
PUT /api/mongo/usuarios/{id}  
DELETE /api/mongo/usuarios/{id}
```

Estudantes MongoDB

```
POST /api/mongo/estudantes  
GET /api/mongo/estudantes  
GET /api/mongo/estudantes/{id}  
PUT /api/mongo/estudantes/{id}  
DELETE /api/mongo/estudantes/{id}
```

Cursos MongoDB

```
POST /api/mongo/cursos  
GET /api/mongo/cursos  
GET /api/mongo/cursos/{id}  
PUT /api/mongo/cursos/{id}  
DELETE /api/mongo/cursos/{id}
```

Vínculos MongoDB

```
POST /api/mongo/vinculos  
GET /api/mongo/vinculos  
GET /api/mongo/vinculos/{id}  
PUT /api/mongo/vinculos/{id}  
DELETE /api/mongo/vinculos/{id}
```

7. Parte 3 — Integração de Dados com PostgreSQL, Esquema Estrela e Apache Hop

7.1 Objetivo

Construir um banco de dados analítico no formato de esquema estrela e desenvolver pipelines de ETL no Apache Hop para carregar as dimensões e a tabela fato.

A integração utilizará:

- banco relacional da Parte 1;
- arquivos CSV de Unidades Acadêmicas;
- arquivos CSV de Componentes Curriculares;
- arquivos CSV de Docentes;
- arquivos CSV de Turmas;
- turmas de 2019 até 2025.

7.2 Modelo Estrela

O esquema estrela será usado para analisar turmas de graduação.

7.3 Dimensões

dim_professor

Campos sugeridos:

- id_professor;
- nome;
- tipo_jornada;
- formação;
- departamento_lotação.

dim_disciplina

Campos sugeridos:

- id_disciplina;
- código;
- nome;
- departamento_responsável;
- cr_total.

dim_departamento

Campos sugeridos:

- id_departamento;
- código ou sigla;
- nome.

dim_semestre

Campos sugeridos:

- id_semestre;
- ano;
- período.

dim_campus

Campos sugeridos:

- id_campus;
- nome.

7.4 Tabela Fato

fato_turma

Campos sugeridos:

- id_fato_turma;
- id_professor;
- id_disciplina;
- id_departamento;
- id_semestre;
- id_campus;
- número de discentes matriculados;
- média de notas;
- número de aprovados;
- número de reprovados.

7.5 Script SQL do Esquema Estrela

```
CREATE TABLE dim_professor (  
    id_professor SERIAL PRIMARY KEY,  
    nome VARCHAR(255) NOT NULL,  
    tipo_jornada VARCHAR(100),  
    formacao VARCHAR(100),  
    departamento_lotacao VARCHAR(100)  
);  
  
CREATE TABLE dim_disciplina (  
    id_disciplina SERIAL PRIMARY KEY,  
    codigo VARCHAR(50) NOT NULL,  
    nome VARCHAR(255) NOT NULL,  
    departamento_responsavel VARCHAR(100),  
    cr_total INTEGER  
);  
  
CREATE TABLE dim_departamento (  
    id_departamento SERIAL PRIMARY KEY,  
    codigo VARCHAR(50) NOT NULL,  
    nome VARCHAR(255) NOT NULL  
);  
  
CREATE TABLE dim_semestre (  
    id_semestre SERIAL PRIMARY KEY,  
    ano INTEGER NOT NULL,  
    periodo INTEGER NOT NULL  
);  
  
CREATE TABLE dim_campus (  
    id_campus SERIAL PRIMARY KEY,  
    nome VARCHAR(255) NOT NULL
```

```
);

CREATE TABLE fato_turma (
    id_fato_turma SERIAL PRIMARY KEY,

    id_professor INTEGER REFERENCES dim_professor(id_professor),
    id_disciplina INTEGER REFERENCES dim_disciplina(id_disciplina),
    id_departamento INTEGER REFERENCES dim_departamento(id_departamento),
    id_semestre INTEGER REFERENCES dim_semestre(id_semestre),
    id_campus INTEGER REFERENCES dim_campus(id_campus),

    numero_discentes_matriculados INTEGER,
    media_notas NUMERIC(5,2),
    numero_aprovados INTEGER,
    numero_reprovados INTEGER
);
```

7.6 Pipelines no Apache Hop

Serão criados pipelines separados para facilitar manutenção e apresentação.

Pipeline 1 — Carga de Departamentos

Fonte:

- CSV de Unidades Acadêmicas.

Destino:

- `dim_departamento`.

Transformações:

- selecionar código/sigla;
- selecionar nome;
- remover duplicidades;
- tratar valores nulos;
- inserir no PostgreSQL.

Pipeline 2 — Carga de Disciplinas

Fonte:

- CSV de Componentes Curriculares.

Destino:

- `dim_disciplina`.

Transformações:

- selecionar código;
- selecionar nome;
- selecionar departamento responsável;
- selecionar créditos;
- converter tipos;
- remover duplicidades;
- inserir no PostgreSQL.

Pipeline 3 — Carga de Professores

Fonte:

- CSV de Docentes.

Destino:

- `dim_professor`.

Transformações:

- selecionar nome;
- selecionar tipo de jornada;
- selecionar formação;
- selecionar departamento de lotação;
- padronizar textos;
- remover duplicidades;
- inserir no PostgreSQL.

Pipeline 4 — Carga de Semestres

Fonte:

- CSV de Turmas.

Destino:

- `dim_semestre`.

Transformações:

- extrair ano;
- extrair período;
- filtrar turmas de 2019 até 2025;
- remover duplicidades;
- inserir no PostgreSQL.

Pipeline 5 — Carga de Campus

Fonte:

- CSV de Turmas.

Destino:

- `dim_campus`.

Transformações:

- extrair campus;
- padronizar nome;
- remover duplicidades;
- inserir no PostgreSQL.

Pipeline 6 — Carga da Tabela Fato

Fontes:

- CSV de Turmas;
- banco relacional da Parte 1;
- dimensões carregadas.

Destino:

- `fato_turma`.

Transformações:

- filtrar turmas de 2019 até 2025;
- buscar chaves das dimensões;
- associar professor;
- associar disciplina;
- associar departamento;
- associar semestre;
- associar campus;
- calcular número de discentes matriculados;
- calcular média de notas, quando disponível;
- calcular número de aprovados;
- calcular número de reprovados;
- inserir na tabela fato.

7.7 Workflow de Execução

Será criado um workflow no Apache Hop para executar os pipelines na ordem correta:

1. carga_departamentos
2. carga_disciplinas
3. carga_professores

4. carga_semestres
5. carga_campus
6. carga_fato_turma

A tabela fato deve ser carregada somente depois das dimensões.

8. Cronograma de Desenvolvimento

Semana 1 — Preparação do Projeto

- Criar repositório no GitHub.
- Criar projeto Spring Boot.
- Configurar Maven.
- Definir estrutura de pacotes.
- Criar README inicial.
- Criar banco PostgreSQL na AWS RDS.
- Testar conexão com PostgreSQL.

Semana 2 — Modelagem Relacional e CRUD de Usuário

- Criar script SQL do PostgreSQL.
- Criar entidade `Usuario`.
- Criar repository `UsuarioRepository`.
- Criar service `UsuarioService`.
- Criar controller `UsuarioController`.
- Testar operações com Postman.
- Registrar evidências no banco.

Semana 3 — CRUD de Estudante, Curso e Vínculo

- Criar entidade `Estudante`.
- Criar entidade `Curso`.
- Criar entidade `Vinculo`.
- Implementar repositories.
- Implementar services.
- Implementar controllers.
- Validar relacionamentos.
- Testar todos os endpoints relacionais.
- Registrar evidências.

Semana 4 — MongoDB e Mapeamento NoSQL

- Criar banco MongoDB na AWS.
- Configurar Spring Data MongoDB.
- Criar documentos MongoDB.
- Criar repositories MongoDB.
- Documentar mapeamento relacional → NoSQL.
- Criar validações de chave, domínio e campos obrigatórios.

Semana 5 — CRUD NoSQL

- Implementar CRUD de usuários no MongoDB.
- Implementar CRUD de estudantes no MongoDB.
- Implementar CRUD de cursos no MongoDB.
- Implementar CRUD de vínculos no MongoDB.
- Validar integridade referencial pela aplicação.
- Testar endpoints `/api/mongo`.
- Registrar evidências no MongoDB Compass.

Semana 6 — Esquema Estrela

- Criar database do esquema estrela no RDS.
- Criar usuário para rotinas de integração.
- Criar tabelas de dimensão.
- Criar tabela fato.
- Testar inserções manuais.
- Criar documentação do modelo dimensional.

Semana 7 — Preparação dos Dados CSV

- Baixar arquivos CSV do grupo Ensino.
- Organizar arquivos na pasta `data`.
- Verificar estrutura dos CSVs.
- Identificar colunas necessárias.
- Filtrar turmas de 2019 até 2025.
- Definir regras de limpeza e padronização.

Semana 8 — Pipelines de Dimensões

- Criar pipeline de departamentos.
- Criar pipeline de disciplinas.
- Criar pipeline de professores.
- Criar pipeline de semestres.
- Criar pipeline de campus.
- Testar execução individual de cada pipeline.
- Validar contagem de registros.

Semana 9 — Pipeline da Tabela Fato

- Criar pipeline de carga da tabela fato.
- Realizar lookups das dimensões.
- Implementar cálculos de métricas.
- Testar carga com base vazia.
- Corrigir inconsistências de dados.
- Registrar evidências.

Semana 10 — Workflow, Relatório e Apresentação

- Criar workflow de execução no Apache Hop.

- Testar execução completa.
- Finalizar relatório.
- Adicionar prints e evidências.
- Preparar roteiro de apresentação.
- Revisar README.
- Fazer simulação da apresentação.

9. Consultas para Demonstração

9.1 Consultas do CRUD Relacional

```
SELECT * FROM usuario;  
SELECT * FROM estudante;  
SELECT * FROM curso;  
SELECT * FROM vinculo;
```

9.2 Consultas do Esquema Estrela

```
SELECT COUNT(*) FROM dim_professor;  
SELECT COUNT(*) FROM dim_disciplina;  
SELECT COUNT(*) FROM dim_departamento;  
SELECT COUNT(*) FROM dim_semestre;  
SELECT COUNT(*) FROM dim_campus;  
SELECT COUNT(*) FROM fato_turma;
```

9.3 Consulta Analítica

```
SELECT  
    s.ano,  
    s.periodo,  
    d.nome AS disciplina,  
    p.nome AS professor,  
    dep.nome AS departamento,  
    c.nome AS campus,  
    f.numero_discentes_matriculados,  
    f.media_notas,  
    f.numero_aprovados,  
    f.numero_reprovados  
FROM fato_turma f  
JOIN dim_semestre s ON s.id_semestre = f.id_semestre  
JOIN dim_disciplina d ON d.id_disciplina = f.id_disciplina  
JOIN dim_professor p ON p.id_professor = f.id_professor  
JOIN dim_departamento dep ON dep.id_departamento = f.id_departamento
```

```
JOIN dim_campus c ON c.id_campus = f.id_campus  
ORDER BY s.ano, s.periodo, d.nome;
```

10. Evidências para Entrega

Para a apresentação, devem ser registradas evidências de:

Parte 1

- criação do banco PostgreSQL na AWS;
- tabelas criadas;
- requisições de criação, leitura, atualização e exclusão;
- consultas SQL mostrando o efeito no banco.

Parte 2

- banco MongoDB na AWS;
- coleções criadas;
- documentos inseridos;
- operações de CRUD funcionando;
- validações de integridade feitas pela aplicação;
- prints do MongoDB Compass.

Parte 3

- tabelas do esquema estrela vazias antes da execução;
- pipelines do Apache Hop;
- execução dos pipelines;
- tabelas preenchidas após execução;
- consultas SQL de contagem;
- consulta analítica final.

11. Checklist Geral

Configuração Inicial

- [] Criar repositório no GitHub.
- [] Criar projeto Java com Spring Boot.
- [] Configurar Maven.
- [] Configurar PostgreSQL.
- [] Configurar MongoDB.
- [] Configurar Apache Hop.
- [] Criar estrutura de pastas.

Parte 1 — PostgreSQL

- ☐ Criar tabelas relacionais.
- ☐ Criar entidades JPA.
- ☐ Criar repositories JPA.
- ☐ Criar services.
- ☐ Criar controllers.
- ☐ Implementar CRUD de usuário.
- ☐ Implementar CRUD de estudante.
- ☐ Implementar CRUD de curso.
- ☐ Implementar CRUD de vínculo.
- ☐ Testar endpoints.
- ☐ Registrar evidências.

Parte 2 — MongoDB

- ☐ Criar coleções.
- ☐ Criar documentos Java.
- ☐ Criar repositories MongoDB.
- ☐ Criar services NoSQL.
- ☐ Criar controllers NoSQL.
- ☐ Implementar CRUD de usuários.
- ☐ Implementar CRUD de estudantes.
- ☐ Implementar CRUD de cursos.
- ☐ Implementar CRUD de vínculos.
- ☐ Implementar validações.
- ☐ Registrar evidências.

Parte 3 — ETL

- ☐ Criar banco do esquema estrela.
- ☐ Criar usuário para integração.
- ☐ Criar dimensões.
- ☐ Criar tabela fato.
- ☐ Baixar CSVs.
- ☐ Criar pipelines de dimensões.
- ☐ Criar pipeline da tabela fato.
- ☐ Criar workflow.
- ☐ Testar execução completa.
- ☐ Registrar evidências.

Documentação

- ☐ Atualizar README.
- ☐ Documentar modelo relacional.
- ☐ Documentar mapeamento NoSQL.
- ☐ Documentar esquema estrela.
- ☐ Documentar pipelines.
- ☐ Criar relatório final.
- ☐ Preparar apresentação.

12. Roteiro de Apresentação

12.1 Introdução

Apresentar o objetivo do trabalho e explicar que o projeto foi desenvolvido em Java com Spring Boot.

12.2 Parte 1

Mostrar:

- estrutura do projeto;
- configuração PostgreSQL;
- endpoints REST;
- execução de CRUD;
- efeito no banco.

12.3 Parte 2

Mostrar:

- mapeamento para MongoDB;
- coleções criadas;
- endpoints NoSQL;
- validações implementadas;
- efeito no MongoDB.

12.4 Parte 3

Mostrar:

- esquema estrela;
- tabelas vazias antes da execução;
- pipelines no Apache Hop;
- execução das rotinas;
- tabelas preenchidas;
- consulta analítica final.

12.5 Conclusão

Explicar que o projeto cumpriu as três etapas: manipulação relacional, manipulação NoSQL e integração de dados para análise em modelo dimensional.

13. Resultado Esperado

Ao final do desenvolvimento, o projeto deverá conter:

- aplicação Java com Spring Boot;
- CRUD relacional em PostgreSQL;
- CRUD NoSQL em MongoDB;
- documentação do mapeamento relacional para NoSQL;
- banco dimensional em esquema estrela;
- pipelines de ETL no Apache Hop;
- evidências de execução;
- relatório final;
- repositório GitHub organizado.